Discover

Define the contract before choosing the model

NexaAssist / LLMOps engineering workshop Session 1 of 5 · 3 hours · source-aligned workshop

Today’s measurable outcomes

- Trace one question across the application layers.
- Verify that trusted documents match the register.
- Propose a gate and explain who approves it.

Three hours with time to build

- 25 min concepts + 20 min code walkthrough
- 90 min pair labs, including a 10 min break
- 25 min gate review + 20 min recap

Showcase / can we trust this policy?

STAFF NEED

Staff need the approved policy explanation.

CONTROLLED FAILURE

A file changes while its register hash stays fixed.

REQUIRED BEHAVIOR

The ingestion boundary must reject that file.

Run the evidence-integrity demonstration

```
python -m training.showcase --session 1
```

Original: 8 documents verified.

Tampered copy: hash mismatch; ingestion refused.

What Is LLMOps, and Why Does It Exist?

Educative lesson 01

Concept → worked example → NexaAssist application

LLMOps manages behavior under real constraints

> Correctness and safety are part of the product.

> Latency shapes whether people can use the service.

> Token consumption determines variable cost.

From a convincing demo to an operated service

A demo tests a few questions in a controlled setting.

Production introduces changing data, users and traffic.

The remedy is a repeatable evaluation and release loop.

A successful request can still be wrong

HTTP 200 tells us the service returned a response.

A valid quote can accompany an unsupported claim.

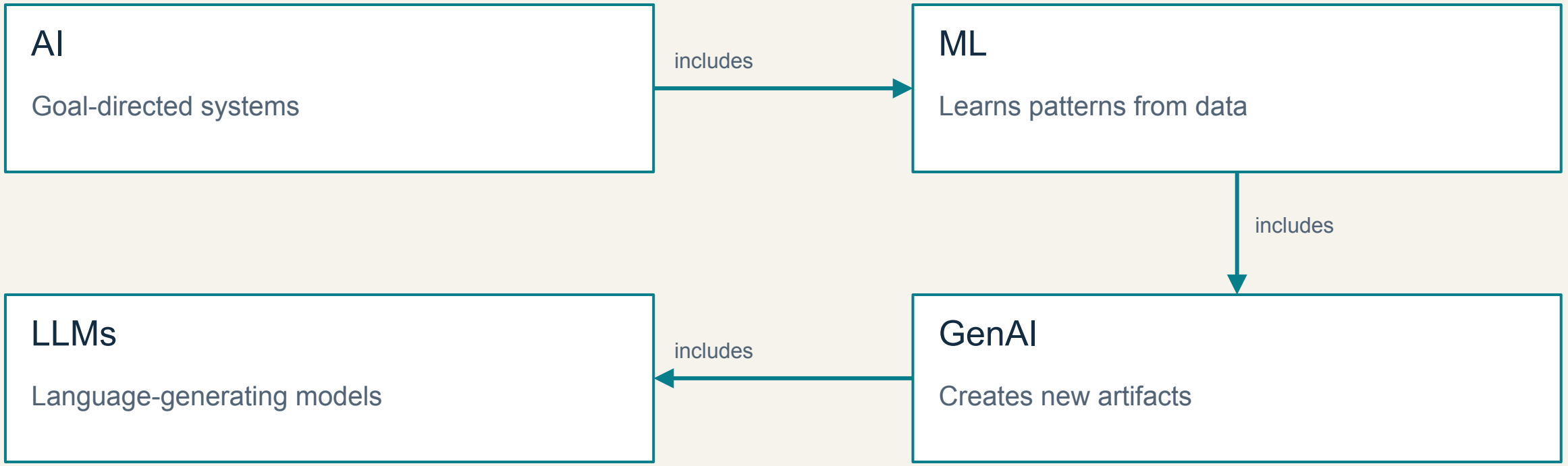
Measure retrieval and meaning alongside availability.

How We Reached LLMOps

Educative lesson 02

Concept → worked example → NexaAssist application

AI, ML, GenAI and LLMs describe different scopes



The operational burden moved toward integration

- Research focused on building and training models.
- Interactive interfaces broadened access to generation.
- Applications now coordinate data, APIs and model behavior.

The artifact gap and the closed-box problem

- Prompts, embeddings and knowledge can change separately.
- Remote model internals are usually outside our control.
- Debug using observed inputs, evidence, outputs and versions.

MLOps to LLMOps: What Changes and What Stays

Educative lesson 03

Concept → worked example → NexaAssist application

Keep the software-engineering foundation

> Version control and CI still coordinate changes.

> Containers and reproducible dependencies still matter.

> Infrastructure configuration still needs review and rollback.

What changes from MLOps to LLMOps

Data

Features → documents + vectors

Tuning

Parameters → prompts + retrieval

Evaluation

Labels → semantic criteria

Artifacts

Weights → coupled runtime assets

Semantic drift needs meaning-aware evidence

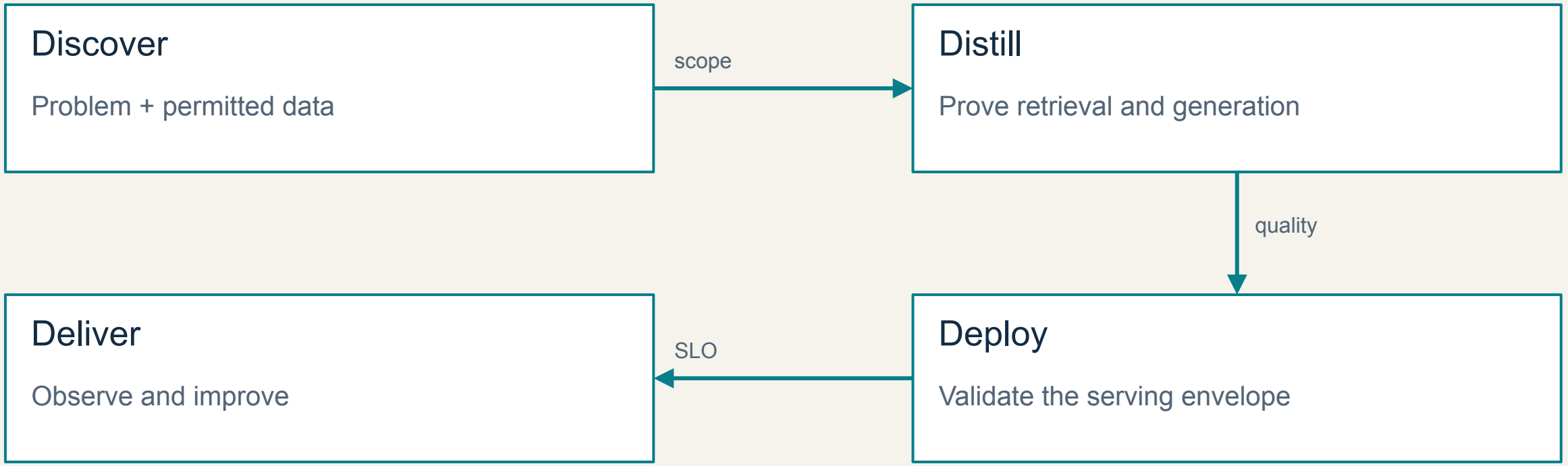
- Different words can express the same request.
- Similar words can refer to different domains.
- Replay representative cases and review emerging topics.

The 4D Framework (Discover, Distill, Deploy, Deliver)

Educative lesson 04

Concept → worked example → NexaAssist application

Four phases, four decisions



Discover produces an operating contract

- Define who asks what and which outcomes are excluded.
- Identify sources, freshness, permissions and disclosure.
- Name proposed quality, latency and cost criteria.

Distill produces evidence, not a favorite prompt

- Tune chunking, retrieval, prompt and model together.
- Use reviewed cases and capture per-case failures.
- Separate a missing source from unsupported generation.

Deploy and Deliver require different evidence

- Deploy: representative traffic within approved budgets.
- Deliver: quality signals, reviewed failures and recovery.
- Missing measurements remain unknown, not successful.

Proposed targets need a rationale

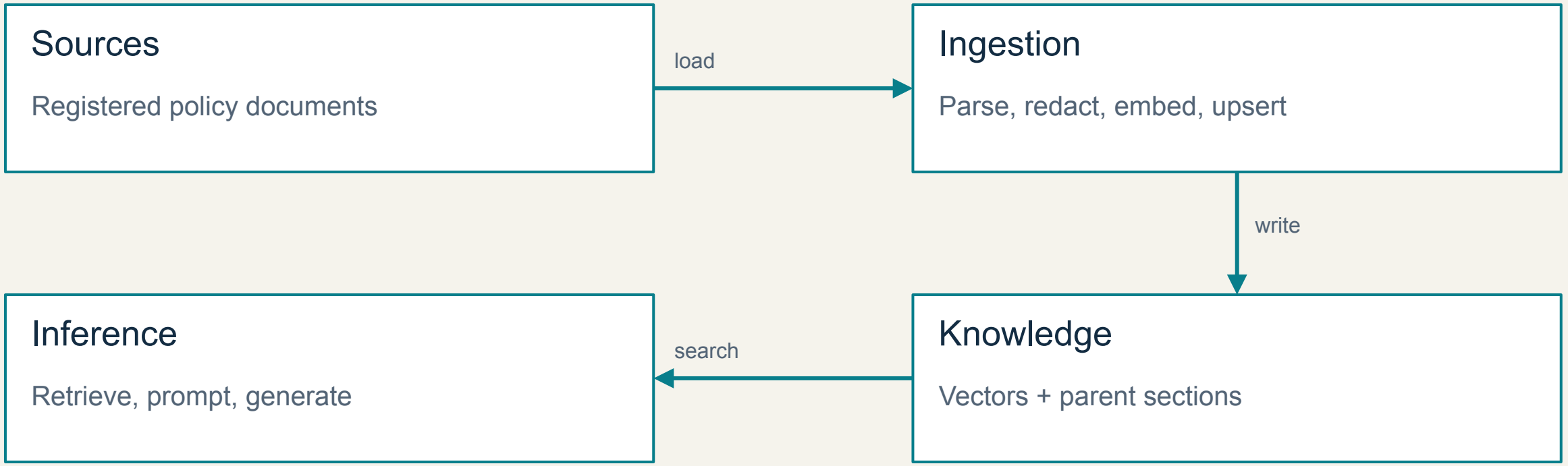
- Quality: source-grounded answers and correct status.
- Operations: latency, error rate and priced cost.
- Safety: a measured rollback signal and named owner.

Architecture of a Retrieval-Augmented Generation Application

Educative lesson 05

Concept → worked example → NexaAssist application

RAG separates knowledge preparation from answering



Freshness, permission and traceability are separate

> A fresh document can still be unauthorized.

> A permitted document can still be obsolete.

> An answer needs a recoverable source identity.

Repeated ingestion needs three guarantees

- Stable identities prevent duplicate accumulation.
- Compatible versions prevent mixed embedding spaces.
- Failure isolation preserves progress and exposes bad files.

Budget the online path before adding hops

Scenario: 2,000 ms total minus 1,500 ms generation.

That leaves 500 ms for queueing, retrieval and overhead.

Parallelism, caching and model choice require measurement.

Use the existing layer boundaries

chains/ owns pure LangChain runnables.

database/ owns MongoDB client and retrieval setup.

services/ coordinates; api/ validates; ui/ renders.

A thin route delegates to the service

```
@router.post("/answer")
async def answer(
    request: RetrieveRequest,
    background: BackgroundTasks,
    settings: Annotated[Settings, Depends(get_settings)],
) -> AnswerResponse:
    try:
        response = await answer_policy(request, settings)
        if should_sample(response, settings):
            background.add_task(score_sampled_answer, request, response, settings)
        return response
    except (
```

Dependency injection supplies settings; the service owns the workflow.

Scoping and Planning a Production RAG Application

Educative lesson 06

Concept → worked example → NexaAssist application

The source course builds an internal support assistant

- Staff search internal policies and tool instructions.
- The response must be grounded and cite its documentation.
- A use-case manifest keeps later scope changes reviewable.

Translate the stack without losing the lesson

- Course: FastAPI, PostgreSQL/pgvector, LiteLLM, Docker.
- NexaAssist: FastAPI, MongoDB Atlas, LangChain, Docker.
- Preserve responsibilities, validation and evaluation gates.

The use-case contract is intentionally narrow

- Explain synthetic policy clauses for staff.
- Ask for missing scope or abstain when unsupported.
- Never approve credit or execute account actions.

A trusted source needs an identity

- Register: doc_id, version, effective dates and hash.
- Authority and amendment relationships affect retrieval.
- A changed file must fail source verification.

Data permission is a release dependency

- Identify who owns and may access the source.
- Decide whether content may leave for embeddings or generation.
- Set freshness and retention before using real documents.

Map the interface boundary honestly

- Course examples describe /ingest and /chat.
- NexaAssist uses an ingestion CLI and /answer routes.
- Operational traces and feedback retain answer context.

Practical 2 / tamper with a safe copy

```
Run: python -m training.session1.tamper
```

Predict which document hash will fail.

Expected: 8 originals verify; changed PDF is refused.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Practical 3 / inspect the gate contract

```
Run: python -m training.session1.check_gates evaluation/gates.json
```

Change a copied faithfulness minimum to 1.5.

Expected: validation rejects a value outside [0, 1].

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Thresholds are typed data

```
class Gates(BaseModel):
    model_config = ConfigDict(extra="forbid")

    # "placeholder" values are course examples that the owner has not yet confirmed.
    status: Literal["placeholder", "confirmed"]
    faithfulness_min: float = Fraction
    answer_relevancy_min: float = Fraction
    context_precision_min: float = Fraction
    context_recall_min: float = Fraction
    judge_correctness_min: float = Fraction
    judge_groundedness_min: float = Fraction
    status_match_rate_min: float = Fraction
```

Status distinguishes placeholder numbers from owner-confirmed targets.

Exit gate / are we ready to experiment?

- Hand in the manifest and data-access decision.
- Name source owner, restrictions and must-abstain cases.
- Keep unresolved approvals visible in the release file.

Check your understanding

> Does a SHA-256 match prove that a policy is correct?

> Can a healthy API answer from the wrong clause?

> Which artifacts must be restored together?

Next / prove retrieval before generation

- Read app/chains/chunking.py.
- Bring a table question and an effective-date trap.
- Carry the manifest into Session 2.